Task 06 - Communicating with AI: Weak -> Better -> Strong

Handbook reference: Page 6 of 10 | Report date: 2026-10-13 | Folder: `task-06-ai-communication/`

1. Concept from the handbook

Reusable structure: Goal -> Context -> Scope -> Constraints -> Acceptance criteria -> Verification -> Required response. A strong request gives the agent both freedom and boundaries. Ask for assumptions, evidence, trade-offs and uncertainty - not for hidden chain-of-thought.

2. What was built

A **PromptSpec** that renders the 7-part structure, a **grader** that scores any free-text prompt 0-7 and lists the missing sections (Weak / Better / Strong), the four copy-ready templates (explain, diagnose, bounded change, review diff), the daily prompt template and a **linter** that flags requests for hidden chain-of-thought.

3. Behaviours demonstrated

- 'Fix the filter.' scores Weak (1/7)
- Observed-vs-expected description scores Better
- Full 7-part prompt scores Strong (7/7)
- Read-only templates explicitly forbid code changes

4. Files in this task

`demo.py`, `prompt_kit.py`, `test_prompt_kit.py`

Run it: `python demo.py` Test it: `python -m unittest discover`

5. Code screenshots

prompt_kit.py

```python
"""Task 06 - Communicating with AI: structured prompts + a prompt grader.

Structure: Goal -> Context -> Scope -> Constraints -> Acceptance criteria
           -> Verification -> Required response
"""
from __future__ import annotations

import re
from dataclasses import dataclass
from typing import Dict, List

SECTIONS = ["goal", "context", "scope", "constraints", "acceptance", "verification", "response"]

# keyword evidence that a section exists in free text
SIGNALS = {
    "goal": r"\b(fix|add|implement|explain|review|diagnose|goal|bug|wrong|shows?)\b",
    "context": r"\b(currently|observed|expected|file|component|because|context)\b",
    "scope": r"\b(scope|only|allowed|limit(ed)? to|files?:)\b",
    "constraints": r"\b(preserve|no unrelated|do not|don't|without|constraint|no new dependenc)",
    "acceptance": r"\b(acceptance|criteria|should|must|expected)\b",
    "verification": r"\b(run|test|check|verify|verification)\b",
    "response": r"\b(report|response|list|show me|give me|diff|assumptions)\b",
}


@dataclass
class PromptSpec:
    goal: str
    context: str
    scope: str
    constraints: str
    acceptance: str
    verification: str
    response: str

    def render(self) -> str:
        return (f"Goal: {self.goal}\nContext: {self.context}\nScope: {self.scope}\n"
                f"Constraints: {self.constraints}\nAcceptance criteria: {self.acceptance}\n"
                f"Verification: {self.verification}\nRequired response: {self.response}")


def grade(prompt: str) -> Dict[str, object]:
    """Score a free-text prompt 0..7 by which sections it covers. Heuristic, not magic."""
    found = {k: bool(re.search(rx, prompt, re.I)) for k, rx in SIGNALS.items()}
    if len(prompt.split()) < 6:                  # too short to carry real context
        found = {k: (k == "goal" and found[k]) for k in found}
    score = sum(found.values())
    level = "Weak" if score <= 2 else "Better" if score <= 5 else "Strong"
    return {"score": score, "of": len(SECTIONS), "level": level,
            "missing": [k for k, v in found.items() if not v]}


TEMPLATES = {
    "explain": "Explain this feature's data flow in simple terms. Name the relevant files/functions, list "
               "assumptions and unclear points separately. Do not modify code.",
    "diagnose": "Compare observed vs expected behavior and test the likely cause with evidence. Do not modify "
                "files. Give the next smallest verification step.",
    "bounded_change": "Make the minimum coherent change for this acceptance criteria. Follow the allowed scope, "
                      "run relevant checks, and report diff + results + limitations.",
    "review_diff": "Review this diff without modifying it. Check correctness, regressions, security/data access "
                   "and missing tests. Give findings with evidence and severity.",
}

DAILY = ("Goal: {task}. Context: {context}. Scope: {scope}. Constraints: preserve existing behavior; no "
         "unrelated refactor. Acceptance: {acceptance}. Verification: run appropriate checks and inspect diff. "
         "Response: assumptions, changes, exact revision/checks/results, checks not run, limitations, next action.")

WEAK = "Fix the filter."
BETTER = "The status filter shows Open tasks when Done is selected. Expected: only Done tasks."
STRONG = PromptSpec(
    goal="Fix the Task Tracker status filter",
    context="Selecting Done also returns Open tasks (stored status is lowercase 'done').",
    scope="filter component + relevant tests only",
    constraints="Preserve existing All behavior; no unrelated refactor; no new dependencies",
    acceptance="All/Open/Done return the right tasks; Done never contains Open",
    verification="Run the Open/Done/All checks and inspect the diff",
    response="Report diff, assumptions, exact checks/results and remaining uncertainty",
).render()

FORBIDDEN_ASKS = [r"chain[- ]of[- ]thought", r"hidden reasoning", r"your (private|internal) thoughts"]
```

Main implementation: prompt_kit.py (part 1/2)

```
81
82
83  def lint_response_request(prompt: str) -> List[str]:
84      """Handbook: ask for concise rationale + verification, NOT hidden chain-of-thought."""
85      return [p for p in FORBIDDEN_ASKS if re.search(p, prompt, re.I)]
```

Main implementation: prompt_kit.py (part 2/2)

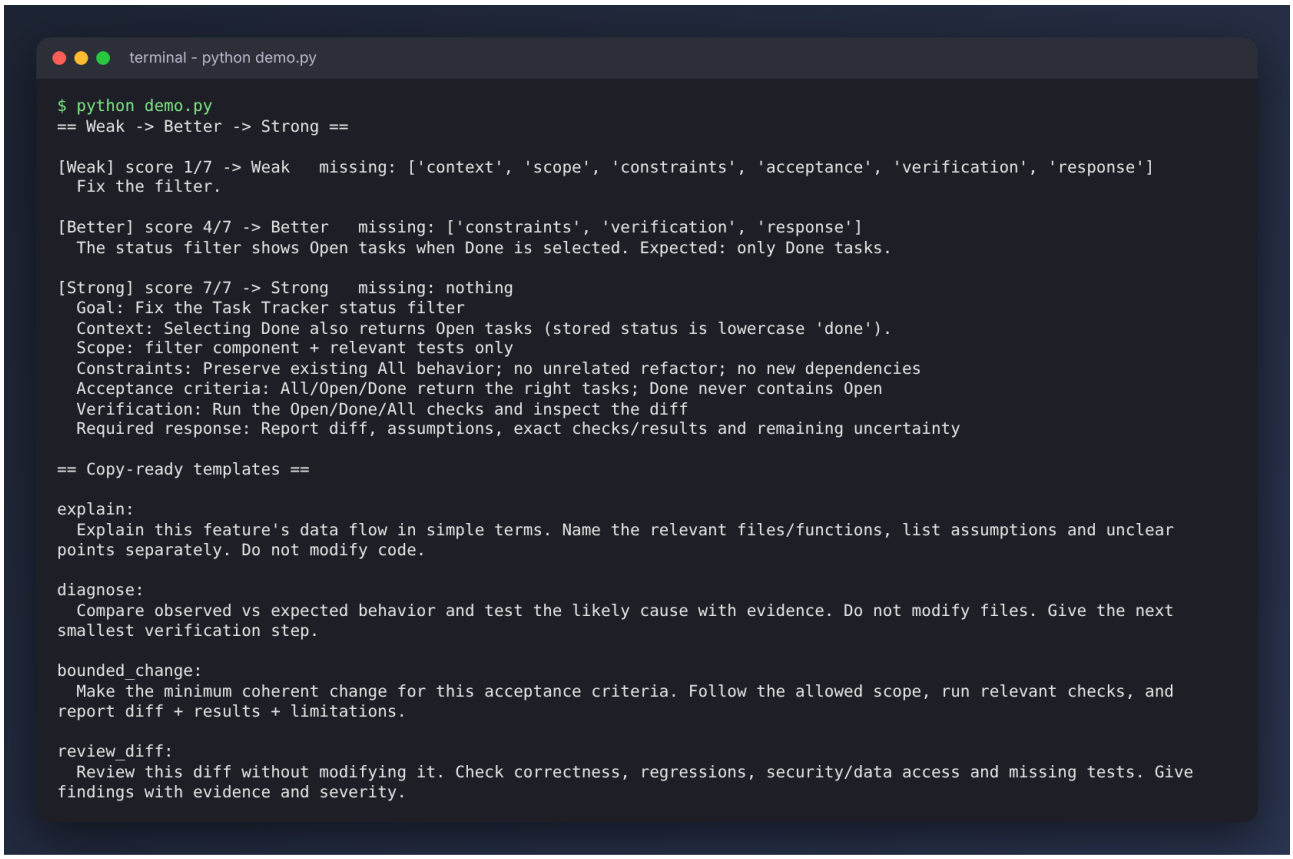


Demo script

6. Demo and test screenshots



Test run (real output)



Demo output (python demo.py)

7. Evidence-backed report

Change	Communicating with AI: Weak -> Better -> Strong
Revision tested	content hash `14d348e6ce22` of all .py files in this folder (re-run with the git commit SHA after you commit)
Checks run	`python -m unittest discover` and `python demo.py`
Results	**PASS** - Ran 7 tests in 0.002s
Checks not run	Sending prompts to a real model
Known limitations	The grader is keyword-based: a quick teaching aid, not a quality guarantee.
Next required action	Human review, then CI on the pull request

8. Practical takeaway

A good prompt makes the task measurable: what to change, what not to, and which evidence means 'done'.